Athel — 03 Discovery and Integrations

Nov 1, 2026 · @Vaibhav Shukla

Athel understands the system by asking the developer's own logged-in tools at the moment it needs an answer, and keeps what it learns only as facts that must be re-checked before use.

Purpose, scope and user stories

This module gives every other module a correct, current picture of the services, data, infrastructure and people involved in a change, with evidence for each claim. It covers features F-04 (scout agent), F-05 (verified knowledge cache) and F-06 (tool integrations).

In scope: reading code across repos; reading Kubernetes, cloud, observability, ticketing, chat and git-host data; combining them into answers; caching facts with verification recipes.

Out of scope: any write to these tools (owned by docs 04, 06, 07); building a permanent service catalog; secrets values (never read).

User stories

ID	As a	I want	So that
US-DISC-01	Developer	to ask "who owns the customer phone number?" and get the owning service with evidence	I change the right service
US-DISC-02	Developer	to know every caller of an endpoint or consumer of an event, with real call volumes	I know who I can break
US-DISC-03	Developer	to see how two services actually talk (protocol, path, ingress, auth)	integration code matches reality
US-DISC-04	Developer	Athel to tell me when what the code says differs from what runs in the cluster	I find drift before it bites
US-DISC-05	Developer	answers to say how sure they are and why	I know when to double-check
US-DISC-06	Developer	repeated questions to be answered fast without stale data	I don't wait or get misled

ID	As a	I want	So that
US-DISC-07	Developer	to know which teammates are working on related code and tickets	I don't collide with them
US-DISC-08	Tech lead	to add a new tool (a new MCP server) without changing Athel code	the framework grows with our stack
US-DISC-09	Developer	current health of the services I will touch (errors, latency, open incidents)	I don't change something already on fire

Functional requirements

Each requirement is testable as written; the acceptance criterion is the test.

ID	Requirement	Acceptance criterion
FR-DISC-001	The scout answers a system question with a claim list, where each claim has at least one source reference (file and line, resource name, dashboard and query, ticket key, message link).	No claim in the output lacks a source; schema validation fails otherwise.
FR-DISC-002	Claims about runtime behaviour (who calls whom, volumes, errors) must come from a runtime source (traces, metrics, logs, mesh or ingress config), not code alone.	A test where code and traffic disagree yields the traffic answer plus a drift note.
FR-DISC-003	Each claim carries a confidence: `confirmed` (two independent sources agree), `likely` (one strong source), `unverified` (inferred).	Claims built only from naming similarity are never `confirmed`.
FR-DISC-004	Before a cached fact is used for any decision, it is re-verified with its recipe unless verified within its freshness window.	Access log shows a verification or an in-window timestamp for 100% of uses.

ID	Requirement	Acceptance criterion
FR-DISC-005	When verification contradicts a cached fact, the fact is superseded (not deleted), and any open change that relied on it is flagged.	A contradiction test produces a superseded record and a flag on the dependent change.
FR-DISC-006	The scout detects and reports drift between sources (code vs cluster, cluster vs traffic, docs vs code).	Fixture with a removed-but-still-called endpoint produces a drift finding.
FR-DISC-007	Every tool used by the scout must be registered in `tools.yaml` with purpose and read verbs; unregistered tools are not called.	Calls to an unregistered MCP tool are denied by the hook.
FR-DISC-008	A new tool is added by a `tools.yaml` entry plus an optional playbook in `skills/integrations/<tool>/`, with no code change.	Adding a fixture MCP server and a playbook makes it usable in a scenario test.
FR-DISC-009	The scout reports which sources it could not reach (auth expired, tool missing) and how that limits the answer.	With Grafana disconnected, the answer states "no runtime traffic data" and lowers confidence.
FR-DISC-010	The scout identifies active work by others on the services in scope: open PRs, recently pushed branches, in-progress tickets, recent commits by others.	Fixture with a teammate's open PR is reported in the "active work" list.
FR-DISC-011	The scout reports current health of in-scope services: error rate, latency percentiles, open alerts or incidents, recent deploys.	Fixture dashboard with a firing alert appears as a health warning.
FR-DISC-012	The scout never reads secret values; it may list secret names and which workloads mount them.	Red-team: any attempt to read a secret value is denied and logged.
FR-DISC-013	Investigation output is saved to `investigation.md` with a machine-readable claim block and the list of fact IDs created or used.	File validates against `schemas/investigation@1`.

ID	Requirement	Acceptance criterion
FR-DISC-014	The scout stops and reports when its question budget (tool calls or time) is used, returning partial findings and gaps.	Budget test returns partial result with gaps listed, not a failure with nothing.

Repo access requirements

ID	Requirement	Acceptance criterion
FR-DISC-015	Athel builds an index of the developer's local clones from confirmed search roots and maps each to its GitHub remote.	Fixture home folder with clones in three different folders produces a complete index.
FR-DISC-016	The developer's local clones are only ever read; Athel never changes their working tree, index, refs or worktree list.	Checksums of each local `.git` folder are unchanged after any scenario.
FR-DISC-017	Repos not available locally are read through the GitHub API or cloned into Athel's cache with the developer's permissions.	A repo missing locally is cloned to the cache; a repo without access is reported as unreachable.
FR-DISC-018	Every Athel write happens in a worktree created from its own cache clone, refreshed from GitHub first.	Scenario shows worktrees only under `workspace/`.

Edge cases and non-functional requirements

Every edge case below has a required behaviour and becomes a scenario test.

Edge case	Required behaviour
Service name differs across sources (`user-svc` in k8s, `users` in code, `user-service` in Grafana)	Resolve aliases with evidence (image name, repo URL in labels, deployment annotations); record the alias set as a fact; never merge on name similarity alone
Monorepo with many services, or one service split across repos	Map services to paths, not only repos; the service-to-code mapping is itself a fact with a recipe

Edge case	Required behaviour
Calls through a gateway, service mesh or message broker	Follow the hop: gateway route config, mesh virtual services, topic subscriptions; report the full path
Dynamic URLs from config or environment variables	Resolve from the deployed config (ConfigMap names, env names) per environment; mark as `likely` if only one environment checked
Events with no schema registry	Infer the payload from producer code; mark consumers found by topic subscription; confidence `likely`
Low-traffic endpoints (zero calls in the lookback window)	Report "no calls seen in N days" rather than "no callers"; extend the window once; check code references too
Traffic data sampled (traces at 1%)	State the sampling rate; never present sampled counts as totals
Multiple clusters, regions or cloud accounts	Every runtime claim names its environment; differences between environments are reported
Tool returns a huge result (thousands of pods, logs)	Query with filters and limits first; save raw output to `artifacts/`; summarise in context
Tool output contains instructions ("ignore previous instructions…")	Treated as data; never followed; flagged in the investigation as suspicious content
Auth expires mid-investigation	Stop that source, keep findings so far, report the gap with the exact login command to run
Conflicting sources with no tie-breaker	Report both with evidence; raise a decision card only if the change depends on the answer
Deleted or renamed services still referenced	Drift finding; never assume the reference is live

Non-functional requirements

ID	Requirement
NFR-DISC-01	A typical ownership or caller question completes in under 3 minutes with warm credentials.
NFR-DISC-02	A cached and still-fresh fact is returned in under 2 seconds.
NFR-DISC-03	Scout output stays under 2,000 words in context; anything larger goes to `artifacts/`.
NFR-DISC-04	Default lookback for traffic is 7 days, configurable; never less than 24 hours.
NFR-DISC-05	All tool calls are read-only and pass the `PreToolUse` policy check (doc 04).
NFR-DISC-06	The scout works with any subset of tools; missing tools reduce confidence, never crash the run.

Scout agent and investigation protocol

The scout is a read-only subagent that answers one question at a time by following a fixed protocol: check the cache, plan the sources, gather, cross-check, and report with confidence.

Agent definition (`agents/scout.md`, summary)

- Tools: file read and search, read-only shell commands on an allow-list (`git log`, `git grep`, `kubectl get`, `kubectl describe`, `gcloud ... list/describe`, `aws ... describe/list`, `gh pr list`, `gh api` GET only, allow-listed praxis read subcommands), read-only MCP tools listed in `tools.yaml`.
- No write tools, no network calls outside registered tools.
- Input: a question, the change ID, the services or repos in scope, a budget.
- Output: `investigation@1` YAML block plus a short Markdown summary.

Protocol per question

1. **Normalise the question** into one of the standard question types (section 7) or `custom`. The type decides which sources are mandatory.
2. **Cache check:** look up facts matching the subject. Fresh facts are used; stale ones are re-verified with their recipe first (section 5).
3. **Source plan:** list sources to query in order of reliability for this question type, skipping unreachable tools and noting them.

4. **Gather** with narrow queries first (filters, label selectors, time windows). Raw results go to `artifacts/`.
5. **Cross-check:** every claim needed for a decision needs two independent sources to be `confirmed`. Code plus runtime counts as independent; two code files do not.
6. **Drift scan:** compare what each source says; any disagreement is a finding, not something to resolve silently.
7. **Report:** claims, confidence, evidence, drift, gaps, active work by others, health. Write new or updated facts to the cache through the helper CLI.

Source reliability by claim type (highest first)

Claim	Sources, most to least reliable
Who calls whom	Traces or mesh telemetry → access logs → gateway or ingress routes → client code → docs
Who owns data	Database schema and migrations in the owning repo → write paths in code → ownership files (CODEOWNERS, catalog) → docs
What is deployed	Cluster or cloud API → deploy pipeline history → manifests in repo
Who owns a service	Catalog or CODEOWNERS → recent committers and reviewers → on-call config → chat channels
Health	Alerting system and dashboards → logs → incident tickets
Active work	Open PRs and branches → in-progress tickets → recent commits → chat mentions

Investigation output (abridged)

```
schema: athel/investigation@1
question: Who consumes the UserUpdated event?
type: event_consumers
claims:
  - id: C1
    text: crm-sync consumes topic users.updated.v1
    confidence: confirmed
    evidence:
      - {kind: code, ref: crm-sync/src/consumers/user.ts:14}
      - {kind: metric, ref: grafana:kafka-consumer-lag?group=crm-sync,
window: 7d}
    facts: [F-0193]
drift:
  - notification-service code subscribes, but no consumer group is active in
prod for 7 days
gaps:
  - Coralogix not authenticated; log evidence missing
active_work:
  - {kind: pr, ref: user-service#812, author: teammate, touches:
src/profile/}
health:
  - {service: crm-sync, env: prod, error_rate_5m: 0.4%, alerts: none}
```

Verified knowledge cache

The cache makes repeated questions fast without ever letting old knowledge drive an action: a fact is a hint until re-verified within its freshness window.

Fact record (`knowledge/facts.jsonl`, one JSON object per line, shown here as YAML)

```
schema: athel/fact@1
id: F-0193
subject: {kind: event, name: UserUpdated, topic: users.updated.v1}
predicate: consumed_by
object: {kind: service, name: crm-sync}
env: prod
confidence: confirmed
sources:
  - {kind: code, ref: crm-sync/src/consumers/user.ts:14, commit: 9f3c2ab}
  - {kind: metric, ref: grafana:kafka-consumer-lag?group=crm-sync}
verify:                     # the recipe: cheap, read-only, deterministic
where possible
  - {tool: git, action: grep, repo: crm-sync, pattern: "users.updated.v1"}
  - {tool: grafana, action: query, expr: "kafka_consumergroup_lag{group='crm-
sync'}", expect: present}
freshness_class: topology    # decides the window
verified_at: 2026-11-01T09:40:00Z
status: active               # active | superseded | disputed
supersedes: null
```

Freshness classes

Class	Examples	Window before re-verify
`volatile`	Health, error rates, alerts, active PRs, who is on call	Always re-verify (0)
`runtime`	Replica counts, deployed image versions, traffic volumes	1 hour
`topology`	Who calls whom, event consumers, ingress routes	24 hours
`structural`	Data ownership, service to repo mapping, aliases	7 days
`organisational`	Team ownership, channels	7 days

Verification rules

1. A fact used for a **decision or a write** is always re-verified if outside its window. A fact used only for orientation may be shown with its age.
2. Verification runs the recipe steps; all must match for `active`. One mismatch makes it `disputed` and triggers a fresh scout pass on that subject.

3. A failed recipe step because a tool is unreachable leaves the fact unchanged but marks the use as `unverified`, which lowers the confidence of anything built on it.
4. Facts are never edited in place. A new verified fact supersedes the old one, keeping history for audit.
5. The helper CLI writes facts; it validates schema, de-duplicates on `(subject, predicate, object, env)`, and refuses facts with no sources or no recipe.
6. Every use appends to `knowledge/access.jsonl`: fact ID, change ID, verified or in-window, purpose.

Human hints: `services.yaml` entries (owners, repo mapping) load as facts with confidence `likely` and a recipe, so human-written knowledge is checked like everything else.

Size and cleanup: facts unused for 90 days are archived; the active file stays small enough to search instantly.

Tool registry and integration catalog

Athel does not re-implement integrations: it uses the MCP servers and CLIs the developer already has, and adds a registry entry that says what each tool is for, which verbs are allowed where, and how to use it well.

Registry entry (`tools.yaml`)

```
- id: kubectl
  kind: cli                        # cli | mcp
  match: {command: "kubectl"}     # for MCP: {server: "grafana", tools:
["*"]}
  purpose: [deployed_state, routing, workload_config, logs]
  read:  [get, describe, logs, top, explain, api-resources]
  write: [apply, delete, scale, rollout, patch, edit, exec, cp, port-forward]
  environments:                    # how to tell which env a call targets
    detect: {flag: "--context", default_from: "kubectl config current-
context"}
  policy:                          # per environment (doc 04 evaluates)
    prod: {read: allow, write: deny}
    staging: {read: allow, write: deny}    # Athel never deploys
    dev: {read: allow, write: deny}
  forbidden_patterns: ["get secret.*-o (yaml|json)", "exec"]
  playbook: skills/integrations/kubectl/SKILL.md
  auth_check: "kubectl auth can-i get pods"
```

Playbook contract (`skills/integrations/<tool>/SKILL.md`): when to use the tool, the 5 to 10 most useful read queries with exact syntax, how to filter large output, known pitfalls, and how to recognise each environment. Playbooks are reference skills: loaded only when the scout needs that tool.

Integration catalog (first set)

Tool	Access	Used for	Reads	Writes (policy in doc 04)
GitHub	`gh` CLI or GitHub MCP	Code search, PRs, branches, CI runs, workflow files, CODEOWNERS	PR list and view, branch list, commits, checks, contents, workflow definitions	Branch push, PR create and comment: change branches only; never merge
Jira or Linear	MCP	Ticket context, owners, related work, status	Issue view and search, comments, links	Comment and status transitions: approval per call
Slack or Teams	MCP	Recent discussion, incidents, who to ask	Channel and thread read, search	Post a message: approval per message, never in incident channels
kubectl	CLI	Deployed state, services, ingress, configs, pod logs	get, describe, logs, top	None in any environment
gcloud	CLI	GKE, Cloud Run, load balancers, Pub/Sub, IAM view, Secret Manager names	list, describe, get-iam-policy	None in any environment
Praxis (Facets)	CLI	Facets view of environments, services, releases and the GCP resources behind them; what a merge will deploy	Allow-listed read subcommands only	None (Praxis is read-only)

Tool	Access	Used for	Reads	Writes (policy in doc 04)
Raptor (Facets)	CLI	Facets infra changes	None	Every call denied; Athel may only write the Raptor command into the deploy guide for the developer to run
AWS CLI	CLI	Only if some services run on AWS	describe-*, list-*, get-* (not secret values)	None
Grafana	MCP	Dashboards, metrics queries, traces, alerts	Dashboard search and read, PromQL and trace queries, alert list	None
Coralogix	MCP	Logs and traces, error search	Log and trace queries	None
Error tracker (Sentry or similar)	MCP	Top errors per service, regressions	Issue list and events	None
Paging (PagerDuty or similar)	MCP	On-call, open incidents	Incident and schedule read	None
GitHub Actions	`gh` CLI	Test results, pipelines, deploy triggers on merge, deploy history	Run list and logs, workflow files	None: Athel never triggers a deploy
Staging database (optional)	Athel credential helper (doc 04)	Verify data after the developer deploys: rows, migration state	`SELECT` only, read-only user	None
Docs (Confluence or Notion)	MCP	Design docs, runbooks	Page search and read	None in v1

Facets layer. Today the infrastructure path is Praxis or Raptor CLI → Facets → GCP. The scout reads Facets through Praxis for the intended setup (services, releases, environment config) and GCP through gcloud and kubectl for what actually runs; a difference between the two is a drift finding. Facets is expected to be retired in 6 to 12 months, so all Facets knowledge sits in its own registry entries and playbook (`skills/integrations/praxis/` plus a deny entry for `raptor`): removing Facets means deleting those entries, with no change to the rest of Athel. The exact Praxis read subcommands go into the allow-list after a review of its help output during milestone M2.

Secret managers appear only as **name listing** (which secrets exist and which workloads mount them). The value-read verbs (`gcloud secrets versions access`, `aws secretsmanager get-secret-value`, `kubectl get secret -o yaml`) are in `forbidden_patterns` for every environment.

Repo access across services

Athel reaches other services' code in the cheapest safe way: the developer's existing local clones first, the GitHub API for quick reads, and its own clone when neither is enough; it never modifies the developer's own clones.

Sources, in order

Order	Source	Used for	Rule
1	Developer's local clones (found by the repo index below)	Fast reading and search; seeding Athel's cache without a full download	Read-only commands only; never checkout, fetch, stash, commit or add worktrees there
2	GitHub API with the developer's login (`gh` or GitHub MCP)	Org-wide code search, single files, PRs and branches for repos not cloned locally	Same permissions as the developer; no access means Athel reports the repo as unreachable

Order	Source	Used for	Rule
3	Athel's own clone in `workspace/cache/<repo>.git`	Deep reading of repos not on disk; the base for every worktree Athel writes in	Partial clone (`--filter=blob:none`), sparse paths for monorepos; seeded from a local clone with `--reference` when one exists; refreshed with `git fetch` before use

Finding local clones (the repo index)

1. At setup, Athel proposes search roots and the developer confirms or edits them once: `~/Developer` , `~/Developers` , `~/Projects` , `~/code` , `~/src` , `~/work` , `~/Documents` , `~/Desktop` , plus any company folder the developer adds (for example `~/snabbit`).
2. The helper CLI walks those roots (depth 4, skipping `node_modules` , `vendor` , build folders) and finds every folder with a `.git` .
3. For each, it reads the remote URL (`git remote get-url origin`) and normalises it (SSH and HTTPS forms become `github.com/org/repo`).
4. Results go to `.athel/repos/index.yaml` : remote, local path, default branch, last commit date. Each entry is a fact with a recipe (the path still exists and still has that remote), re-checked before use.
5. The index refreshes on demand and at session start if older than 7 days; a repo not in the index is looked up on GitHub and cloned into the cache when needed.

Mapping a service to its repo

The scout starts from what runs, not from names: the container image in Kubernetes or Facets points to the GitHub Actions workflow that built it, which points to the repo. CODEOWNERS, org code search and `services.yaml` hints fill gaps. Each mapping is a `structural` fact (see the cache section above).

Rules and edge cases

Case	Behaviour
Local clone is behind GitHub	Read code from Athel's refreshed cache, not the stale working copy; mention that the local clone is behind

Case	Behaviour
Local clone has uncommitted changes	Run read-only `git status` only to report "you have local changes in these files" as possible active work; never read or use the uncommitted content for decisions
Same remote cloned twice locally	Prefer the most recently used path for reading; the cache is still the base for writes
A fork instead of the org repo	Match on the org remote; a fork is used only if the developer says so
Monorepo	One cache clone with sparse checkout of the services' folders
No access on GitHub and no local clone	Report the gap; blast radius confidence drops for that consumer
Search roots include personal folders	Only folders with a `.git` are recorded; file contents outside repos are never read

Question recipes, failure modes and tests

Ten standard question types cover most of what other modules ask; each has mandatory sources, so an answer missing one of them is automatically marked lower confidence.

Type	Example	Mandatory sources	Optional sources
`data_owner`	Who owns customer phone number?	Schemas and migrations; write paths in code	Catalog, docs
`endpoint_callers`	Who calls `PUT /users/{id}`?	Traces or access logs; client code search across repos	Gateway routes
`event_consumers`	Who consumes `UserUpdated`?	Subscription code; consumer group or subscription metrics	Schema registry
`service_path`	How does order-service reach payments?	Client code; cluster service and ingress or mesh config	Traces

Type	Example	Mandatory sources	Optional sources
`deployed_state`	What version of user-service runs in staging?	Cluster or cloud API	Deploy history
`service_health`	Is crm-sync healthy?	Metrics or alerts	Logs, error tracker, incidents
`service_owner`	Which team owns notification-service?	CODEOWNERS or catalog	Recent reviewers, on-call
`active_work`	Is anyone changing user-service's profile code?	Open PRs and branches	Tickets, chat
`config_dependency`	Which config and secrets does payments read?	Deployment env and mounts (names only)	Code
`drift_check`	Does what runs match the repo?	Cluster state; manifests in repo	Deploy history

Failure modes

Failure	Detection	Handling
Wrong alias merge (two services treated as one)	Conflicting runtime evidence for the merged entity	Split; mark the alias fact disputed; re-run dependent questions
Missed consumer (no traffic in window, no code reference found)	Post-merge incident or UAT finding	Add a lesson; extend recipe with the source that would have caught it
Over-broad queries exhausting budget	Budget warnings	Playbooks require filtered queries first; scout returns partial results
Cache poisoning via injected tool output	Fact sources must be tool references, not free text; verification re-derives from source	Disputed fact; security finding

Failure	Detection	Handling
Tool rate limits	`E-TOOL` 429s	Backoff; prefer cache; reduce query count

Tests

- Fixture system `tests/fixtures/shop/` with deliberate traps: an alias mismatch, a dead endpoint still in code, an event consumer present only in another repo, a sampled-trace service, a gateway hop, and an injected instruction in a log line.
- Each question type has at least two scenario tests (one clean, one trap); pass bar 100% for confidence labelling, 95% for claim correctness.
- Cache tests: freshness windows, supersede chain, disputed flow, access log completeness.
- Integration contract tests: a mock MCP server and mock CLIs prove a new tool can be added by registry entry only (FR-DISC-008).